Cortex-M3/M4 Instructions

Instruction	Operands	Description and Action
ADC, ADCS	{Rd,} Rn, Op2	Add with Carry, Rd ← Rn + Op2 + Carry, ADCS updates N,Z,C,V
ADD, ADDS	{Rd,} Rn, Op2	Add, Rd ← Rn + Op2, ADDS updates N,Z,C,V
ADD, ADDS	{Rd,} Rn, #imm12	Add Immediate, Rd ← Rn + imm12, ADDS updates N,Z,C,V
ADR	Rd, label	Load PC-relative Address, Rd ← <label>
AND, ANDS	{Rd,} Rn, Op2	Logical AND, Rd ← Rn AND Op2, ANDS updates N,Z,C
ASR, ASRS	Rd, Rm, <Rs|#n>	Arithmetic Shift Right, Rd ← Rm>>(Rs|n), ASRS updates N,Z,C
B	label	Branch, PC ← label
BFC	Rd, #lsb, #width	Bit Field Clear, Rd[(width+lsb–1):lsb] ← 0
BFI	Rd, Rn, #lsb, #width	Bit Field Insert, Rd[(width+lsb–1):lsb] ← Rn[(width-1):0]
BIC, BICS	{Rd,} Rn, Op2	Bit Clear, Rd ← Rn AND NOT Op2, BICS updates N,Z,C
BKPT	#imm	Breakpoint, prefetch abort or enter debug state
BL	label	Branch with Link, LR ← address of next instruction, PC ← label
BLX	Rm	Branch register with link, LR ← address of next instruction, PC ← Rm[31:1]
BX	Rm	Branch register, PC ← Rm
CBNZ	Rn, label	Compare and Branch if Non-zero; PC ← label if Rn != 0
CBZ	Rn, label	Compare and Branch if Zero; PC ← label if Rn == 0
CLREX	-	Clear local processor exclusive tag
CLZ	Rd, Rm	Count Leading Zeroes, Rd ← number of leading zeroes in Rm
CMN	Rn, Op2	Compare Negative, Update N,Z,C,V flags on Rn + Op2
CMP	Rn, Op2	Compare, Update N,Z,C,V flags on Rn − Op2
CPSID	i	Disable specified (i) interrupts, optional change mode
CPSIE	i	Enable specified (i) interrupts, optional change mode
DMB	-	Data Memory Barrier, ensure memory access order
DSB	-	Data Synchronization Barrier, ensure completion of access
EOR, EORS	{Rd,} Rn, Op2	Exclusive OR, Rd ← Rn XOR Op2, EORS updates N,Z,C
ISB	-	Instruction Synchronization Barrier
IT	-	If-Then Condition Block
LDM	Rn{!}, reglist	Load Multiple Registers increment after, <reglist> = mem[Rn], Rn increments after each memory access
LDMDB, LDMEA	Rn{!}, reglist	Load Multiple Registers Decrement Before, <reglist> = mem[Rn], Rn decrements before each memory access
LDMFD, LDMIA	Rn{!}, reglist	<reglist> = mem[Rn], Rn increments after each memory access
LDR	Rt, [Rn, #offset]	Load Register with Word, Rt ← mem[Rn + offset]
LDRB, LDRBT	Rt, [Rn, #offset]	Load Register with Byte, Rt ← mem[Rn + offset]
LDRD	Rt, Rt2, [Rn,#offset]	Load Register with two words, Rt ← mem[Rn + offset], Rt2 ← mem[Rn + offset + 4]
LDREX	Rt, [Rn, #offset]	Load Register Exclusive, Rt ← mem[Rn + offset]
LDREXB	Rt, [Rn]	Load Register Exclusive with Byte, Rt ← mem[Rn]
LDREXH	Rt, [Rn]	Load Register Exclusive with Halfword, Rt ← mem[Rn]
LDRH, LDRHT	Rt, [Rn, #offset]	Load Register with Halfword, Rt ← mem[Rn + offset]
LDRSB, LDRSBT	Rt, [Rn, #offset]	Load Register with Signed Byte, Rt ← mem[Rn + offset]
LDRSH, LDRSHT	Rt, [Rn, #offset]	Load Register with Signed Halfword, Rt ← mem[Rn + offset]
LDRT	Rt, [Rn, #offset]	Load Register with Word, Rt ← mem[Rn + offset]
LSL, LSLS	Rd, Rm, <Rs|#n>	Logic Shift Left, Rd ← Rm << Rs|n, LSLS update N,Z,C
LSR, LSRS	Rd, Rm, <Rs|#n>	Logic Shift Right, Rd ← Rm >> Rs|n, LSRS update N,Z,C
MLA	Rd, Rn, Rm, Ra	Multiply with Accumulate, Rd ← (Ra + (Rn*Rm))[31:0]
MLS	Rd, Rn, Rm, Ra	Multiply with Subtract, Rd ← (Ra – (Rn*Rm))[31:0]
MOV, MOVS	Rd, Op2	Move, Rd ← Op2, MOVS updates N,Z,C
MOVT	Rd, #imm16	Move Top, Rd[31:16] ← imm16, Rd[15:0] unaffected
MOVW, MOVWS	Rd, #imm16	Move 16-bit Constant, Rd ← imm16, MOVWS updates N,Z,C
MRS	Rd, spec_reg	Move from Special Register, Rd ← spec_reg
MSR	spec_reg, Rm	Move to Special Register, spec_reg ← Rm, Updates N,Z,C,V
MUL, MULS	{Rd,} Rn, Rm	Multiply, Rd ← (Rn*Rm)[31:0], MULS updates N,Z
MVN, MVNS	Rd, Op2	Move NOT, Rd ← 0xFFFFFFFF EOR Op2, MVNS updates N,Z,C
NOP	-	No Operation
ORN, ORNS	{Rd,} Rn, Op2	Logical OR NOT, Rd ← Rn OR NOT Op2, ORNS updates N,Z,C
ORR, ORRS	{Rd,} Rn, Op2	Logical OR, Rd ← Rn OR Op2, ORRS updates N,Z,C
POP	reglist	Canonical form of LDM SP!, <reglist>
PUSH	reglist	Canonical form of STMDB SP!, <reglist>
RBIT	Rd, Rn	Reverse Bits, for (i = 0; i < 32; i++): Rd[i] = RN[31–i]
REV	Rd, Rn	Reverse Byte Order in a Word, Rd[31:24]←Rn[7:0], Rd[23:16]←Rn[15:8], Rd[15:8]←Rn[23:16], Rd[7:0]←Rn[31:24]
REV16	Rd, Rn	Reverse Byte Order in a Halfword, Rd[15:8]←Rn[7:0], Rd[7:0]←Rn[15:8], Rd[31:24]←Rn[23:16], Rd[23:16]←Rn[31:24]

REVSH	Rd, Rn	Reverse Byte order in Low Halfword and sign extend, Rd[15:8]←Rn[7:0], Rd[7:0]←Rn[15:8], Rd[31:16]←Rn[7]*&FFFF
ROR, RORS	Rd, Rm, <Rs|#n>	Rotate Right, Rd ← ROR(Rm, Rs|n), RORS updates N,Z,C
RRX, RRXS	Rd, Rm	Rotate Right with Extend, Rd ← RRX(Rm), RRXS updates N,Z,C
RSB, RSBS	{Rd,} Rn, Op2	Reverse Subtract, Rd ← Op2 − Rn, RSBS updates N,Z,C,V
SBC, SBCS	{Rd,} Rn, Op2	Subtract with Carry, Rd ← Rn−Op2−NOT(Carry), updates NZCV
SBFX	Rd, Rn, #lsb, #width	Signed Bit Field Extract, Rd[(width−1):0] = Rn[(width+lsb−1):lsb], Rd[31:width] = Replicate(Rn[width+lsb−1])
SDIV	{Rd,} Rn, Rm	Signed Divide, Rd ← Rn/Rm
SEV	-	Send Event
SMLAL	RdLo, RdHi, Rn, Rm	Signed Multiply with Accumulate, RdHi,RdLo ← signed(RdHi,RdLo + Rn*Rm)
SMULL	RdLo, RdHi, Rn, Rm	Signed Multiply, RdHi,RdLo ← signed(Rn*Rm)
SSAT	Rd, #n, Rm{,shift #s}	Signed Saturate, Rd ← SignedSat((Rm shift s), n). Update Q
STM	Rn{!}, reglist	Store Multiple Registers
STMDB, STMEA	Rn{!}, reglist	Store Multiple Registers Decrement Before
STMFD, STMIA	Rn{!}, reglist	Store Multiple Registers Increment After
STR	Rt, [Rn, #offset]	Store Register with Word, mem[Rn+offset] = Rt
STRB, STRBT	Rt, [Rn, #offset]	Store Register with Byte, mem[Rn+offset] = Rt
STRD	Rt, Rt, [Rn,#offset]	Store Register with two Words, mem[Rn+offset] = Rt, mem[Rn+offset+4] = Rt2
STREX	Rd, Rt, [Rn,#offset]	Store Register Exclusive, If allowed, mem[Rn + offset] ← Rt, clear exclusive tag, Rd ← 0. Else Rd ← 1.
STREXB	Rd, Rt, [Rn]	Store Register Exclusive Byte, mem[Rn] ← Rt[15:0] or mem[Rn] ← Rt[7:0], clear exclusive tag, Rd ← 0. Else Rd ← 1
STREXH	Rd, Rt, [Rn]	Store Register Exclusive Halfword, mem[Rn] ← Rt[15:0] or mem[Rn] ← Rt[7:0], clear exclusive tag, Rd ← 0. Else Rd ← 1
STRH, STRHT	Rt, [Rn, #offset]	Store Halfword, mem[Rn + offset] ← Rt[15:0]
STRT	Rt, [Rn, #offset]	Store Register with Translation, mem[Rn + offset] = Rt
SUB, SUBS	{Rd,} Rn, Op2	Subtraction, Rd ← Rn − Op2, SUBS updates N,Z,C,V
SUB, SUBS	{Rd,} Rn, #imm12	Subtraction, Rd ← Rn-imm12, SUBS updates N,Z,C,V
SVC	#imm	Supervisor Call
SXTB	{Rd,} Rm {,ROR #n}	Sign Extend Byte, Rd ← SignExtend((Rm ROR (8*n))[7:0])
SXTH	{Rd,} Rm {,ROR #n}	Sign Extend Halfword, Rd ← SignExtend((Rm ROR (8*n))[15:0])
TBB	[Rn, Rm]	Table Branch Byte, PC ← PC+ZeroExtend(Memory(Rn+Rm,1)<<1)
TBH	[Rn, Rm, LSL #1]	Table Branch Halfword, PC ← PC + ZeroExtend(Memory(Rn+Rm<<1, 2)<<1)
TEQ	Rn, Op2	Test Equivalence, Update N,Z,C,V on Rn EOR Operand2
TST	Rn, Op2	Test, Update N,Z,C,V on Rn AND Op2
UBFX	Rd, Rn, #lsb, #width	Unsigned Bit Field Extract, Rd[(width−1):0] = Rn[(width+lsb−1):lsb], Rd[31:width] = Replicate(0)
UDIV	{Rd,} Rn, Rm	Unsigned Divide, Rd ← Rn/Rm
UMLAL	RdLo, RdHi, Rn, Rm	Unsigned Multiply with Accumulate, RdHi,RdLo ← unsigned(RdHi,RdLo + Rn*Rm)
UMULL	RdLo, RdHi, Rn, Rm	Unsigned Multiply, RdHi,RdLo ← unsigned(Rn*Rm)
USAT	Rd, #n, Rm{,shift #s}	Unsigned Saturate, Rd←UnsignedSat((Rm shift s),n), Update Q
UXTB	{Rd,} Rm {,ROR #n}	Unsigned Extend Byte, Rd ← ZeroExtend((Rm ROR (8*n))[7:0])
UXTH	{Rd,} Rm {,ROR #n}	Unsigned Extend Halfword, Rd ← ZeroExtend((Rm ROR (8*n))[15:0])
WFE	-	Wait For Event
WFI	-	Wait for Interrupt

Conditional Codes

Compare	Signed	Unsigned
>	BGT	BHI
>=	BGE	BHS
<	BLT	BLO
<=	BLE	BLS
==	BEQ	
!=	BNE	

FPU Instructions

Instruction	Operands	Description and Action
VABS.F32	Sd, Sm	Absolute value of floats, Sd ← |Sm|
VADD.F32	{Sd,} Sn, Sm	Add floating points, Sd ← Sn + Sm
VCMP.F32	Sd, <Sm | #0.0>	Compare two floating-point registers, or one floating-point register and zero
VCMPE.F32	Sd, <Sm | #0.0>	Compare two floating-point registers, or one floating-point register and zero, and raise exception for a signaling NaN
VCVT{R}.S32.F32	Sd, Sm	Convert from single-precision to signed 32-bit (S32) or unsigned 32-bit (U32) integer. If R is specified, it uses the rounding mode specified by FPSCR. If R is omitted, it uses round towards zero.
VCVT{R}.U32.F32	Sd, Sm	
VCVT{R}.F32.S32	Sd, Sm	Convert to single-precision from signed 32-bit (S32) or unsigned 32-bit (U32) integer. See above for R.
VCVT{R}.F32.U32	Sd, Sm	
VCVT{R}.Td.F32	Sd, Sm, #fbits	Convert between single-precision and fixed-point. Td can be S16 (signed 16-bit), U16 (unsigned 16-bit), S32 (signed 32-bit), U32 (unsigned 32-bit). fbits is the number of fraction bits in the fixed-point number. See above for R.
VCVT{R}.Td.F32	Sd, Sd, #fbits	
VCVT{R}.F32.Td	Sd, Sm, #fbits	
VCVT{R}.F32.Td	Sd, Sd, #fbits	
VCVT<B|T>.F32.F16	Sd, Sm	Converts half-precision float to single-precision (B = bottom half of Sm, T = top half of Sm)
VCVT<B|T>.F16.F32	Sd, Sm	Converts single-precision float to half-precision (B = bottom half of Sd, T = top half of Sd)
VDIV.F32	{Sd,} Sn, Sm	Divide single-precision floats, Sd = Sn/Sm
VFMA.F32	{Sd,} Sn, Sm	Multiply (fused) then accumulate float, Sd = Sd + Sn*Sm
VFMS.F32	{Sd,} Sn, Sm	Multiply (fused) then subtract float, Sd = Sd - Sn*Sm
VFNMA.F32	{Sd,} Sn, Sm	Multiply (fused) then accumulate then negate float, Sd = -1 * Sd + Sn * Sm
VFNMS.F32	{Sd,} Sn, Sm	Multiply (fused) then subtract then negate float, Sd = -1 * Sd - Sn * Sm
VLDM.64	Rn{!}, list	Load multiple double-precision floats
VLDM.32	Rn{!}, list	Load multiple single-precision floats
VLDR.F64	<Dd|Sd>, [Rn]	Load one double-precision float
VLDR.F32	<Dd|Sd>, [Rn]	Load one single-precision float
VLMA.F32	{Sd,} Sn, Sm	Multiply float then accumulate float, Sd = Sd + Sn*Sm
VLMS.F32	{Sd,} Sn, Sm	Multiply float then subtract float, Sd = Sd - Sn*Sm
VMOV.F32	Sd, #imm	Move immediate to float-register
VMOV	Sd, Sm	Copy from float register to float register
VMOV	Sn, Rt	Copy ARM core register to float register
VMOV	Sm, Sm1, Rt, Rt2	Copy 2 ARM core registers to 2 float registers
VMOV	Dd[x], Rt	Copy ARM core register to a half of a double-precision floating-point register, where x is 0 or 1.
VMOV	Rt, Dn[x]	Copy a half of a double-precision floating-point register to ARM core register, where x is 0 or 1.
VMRS	Rt, FPSCR	Move FPSCR to ARM core register or APSR
VMSR	FPSCR, Rt	Move to FPSCR from ARM Core register
VMUL.F32	{Sd,} Sn, Sm	Multiply float, Sd = Sn * Sm
VNEG.F32	Sd, Sm	Negate float, Sd = -1 * Sm
VNMLA.F32	Sd, Sn, Sm	Multiply float then accumulate then negate float Sd = -1 * (Sd + Sn * Sm)
VNMLS.F32	Sd, Sn, Sm	Multiply float then subtract then negate float Sd = -1 * (Sd - Sn * Sm)
VNMUL.F32	{Sd,} Sn, Sm	Negate and multiply float, Sd = -1 * Sn * Sm
VPOP.64	list	Pop double registers from stack
VPOP.32	list	Pop float registers from stack
VPUSH.64	list	Push double registers to stack
VPUSH.32	list	Push float registers to stack
VSQRT.F32	Sd, Sm	Square-root of float
VSTM.64	Rn{!}, list	Store multiple double registers
VSTM.32	Rn{!}, list	Store multiple float registers
VSTR.64	Sd, [Rn]	Store one double register
VSTR.32	Sd, [Rn]	Store one float registers
VSUB.F32	{Sd,} Sn, Sm	Subtract float, Sd = Sn - Sm

Some of the register definitions:

1. GPIO port mode register (GPIOx_MODER) (x =A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODE[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O mode.
00: Input mode
01: General purpose output mode
10: Alternate function mode
11: Analog mode (reset state)

2. GPIO port output type register (GPIOx_OTYPER) (x = A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **OT[15:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O output type.
0: Output push-pull (reset state)
1: Output open-drain

3. GPIO port output speed register (GPIOx_OSPEEDR) (x = A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEED15 [1:0]		OSPEED14 [1:0]		OSPEED13 [1:0]		OSPEED12 [1:0]		OSPEED11 [1:0]		OSPEED10 [1:0]		OSPEED9 [1:0]		OSPEED8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEED7 [1:0]		OSPEED6 [1:0]		OSPEED5 [1:0]		OSPEED4 [1:0]		OSPEED3 [1:0]		OSPEED2 [1:0]		OSPEED1 [1:0]		OSPEED0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **OSPEED[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O output speed.
00: Low speed
01: Medium speed
10: High speed
11: Very high speed

4. GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPD15[1:0]		PUPD14[1:0]		PUPD13[1:0]		PUPD12[1:0]		PUPD11[1:0]		PUPD10[1:0]		PUPD9[1:0]		PUPD8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPD7[1:0]		PUPD6[1:0]		PUPD5[1:0]		PUPD4[1:0]		PUPD3[1:0]		PUPD2[1:0]		PUPD1[1:0]		PUPD0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PUPD[15:0][1:0]:** Port x configuration I/O pin y (y = 15 to 0)
These bits are written by software to configure the I/O pull-up or pull-down
00: No pull-up, pull-down
01: Pull-up
10: Pull-down
11: Reserved

5. GPIO port input data register (GPIOx_IDR) (x = A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ID15	ID14	ID13	ID12	ID11	ID10	ID9	ID8	ID7	ID6	ID5	ID4	ID3	ID2	ID1	ID0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

6. GPIO port output data register (GPIOx_ODR) (x = A to I)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

7. AHB2 peripheral clock enable register (RCC_AHB2ENR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	HASHE N	AESEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	DCMIE N	ADCEN	OTGFS EN	Res.	Res.	Res.	GPIOIE N	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
	rw	rw	rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 0 **GPIOAEN:** IO port A clock enable
Set and cleared by software.
0: IO port A clock disabled
1: IO port A clock enabled